

Example 1: Handling ArithmeticException (Division by Zero)

```
import java.util.Scanner;

public class ExceptionDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
            System.out.print("Enter first number: ");
            int a = sc.nextInt();

            System.out.print("Enter second number: ");
            int b = sc.nextInt();

            int result = a / b; // May cause ArithmeticException

            System.out.println("Result = " + result);
        }
        catch (ArithmeticException e) {
            System.out.println("Error: Cannot divide by zero.");
        }
        catch (Exception e) {
            System.out.println("Invalid input.");
        }
        finally {
            System.out.println("Program completed.");
        }

        sc.close();
    }
}
```

Example 2 : Handling multiple exceptions

```
public class MultipleExceptionDemo {
    public static void main(String[] args) {

        try {
            int arr[] = new int[3];
            arr[5] = 10; // ArrayIndexOutOfBoundsException

            int x = 10 / 0; // ArithmeticException
        }
        catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Array index is invalid.");
        }
        catch (ArithmeticException e) {
```

```

        System.out.println("Cannot divide by zero.");
    }
    catch (Exception e) {
        System.out.println("Some other exception occurred.");
    }

    System.out.println("Program continues...");
}
}

```

Example 3: User Defined Exceptions

```

class AgeException extends Exception {
    AgeException(String message) {
        super(message);
    }
}

```

```

public class CustomExceptionDemo {
    public static void main(String[] args) {

        int age = 16;

        try {
            if (age < 18) {
                throw new AgeException("Not eligible to vote.");
            }
            else {
                System.out.println("Eligible to vote.");
            }
        }
        catch (AgeException e) {
            System.out.println("Exception: " + e.getMessage());
        }
    }
}

```

Throw Demo

```

class ThrowDemo {

    public static void main(String[] args) {

        try {
            int age = 16;

            if (age < 18) {

```

```

        throw new ArithmeticException("Not eligible to vote");
    }

    System.out.println("Eligible to vote");
}

catch (ArithmeticException e) {
    System.out.println("Exception caught: " + e.getMessage());
}

finally {
    System.out.println("Program execution completed.");
}
}
}

```

Throws keyword

- Used in **method declaration**
- Declares that a method **may throw an exception**
- Mainly used for **checked exceptions**
- Responsibility of handling is given to the calling method

```

class ArithmeticThrowsExample {

    // Method declaring ArithmeticException using throws

    static int divide(int a, int b) throws ArithmeticException {
        return a / b; // May cause ArithmeticException
    }

    public static void main(String[] args) {

        try {
            int result = divide(10, 0); // Calling method
            System.out.println("Result: " + result);
        }

        catch (ArithmeticException e) {
            System.out.println("Exception caught: Cannot divide by zero.");
        }

        System.out.println("Program ended.");
    }
}

```

Throws usually used with checked exceptions

```
import java.io.FileReader;
import java.io.IOException;

class ThrowsExample {

    // Method declaring exception using throws
    static void readFile() throws IOException {
        FileReader file = new FileReader("abc.txt");
        System.out.println("File opened successfully.");
    }

    public static void main(String[] args) {

        try {
            readFile(); // Calling method
        }

        catch (IOException e) {
            System.out.println("Exception handled in main: File not found.");
        }

        System.out.println("Program ended.");
    }
}
```